Graph Data Structure Study Guide

1. Topological Sort / Kahn's Algorithm

What is Topological Sort?

Topological sorting is a linear ordering of vertices in a Directed Acyclic Graph (DAG) such that for every directed edge (u, v), vertex u comes before vertex v in the ordering. This is only possible in DAGs (graphs with no cycles).

Real-world applications:

- Course prerequisites in university
- Task scheduling with dependencies
- Build systems and dependency resolution

Kahn's Algorithm

Kahn's algorithm uses BFS approach to find topological ordering by repeatedly removing vertices with no incoming edges.

Algorithm Steps:

1. Calculate in-degree for all vertices
2. Add all vertices with in-degree 0 to a queue
3. While queue is not empty:
    - Remove a vertex from queue
    - Add it to result
    - For each neighbor, decrease its in-degree by 1
    - If neighbor's in-degree becomes 0, add to queue
4. If result contains all vertices, return result; otherwise, graph has a cycle

Java Implementation

java

```java
import java.util.*;

public class TopologicalSort {

    // Graph representation using adjacency list
    private Map<Integer, List<Integer>> graph;
    private int vertices;

    public TopologicalSort(int vertices) {
        this.vertices = vertices;
        this.graph = new HashMap<>();
        for (int i = 0; i < vertices; i++) {
            graph.put(i, new ArrayList<>());
        }
    }

    // Add directed edge from u to v
    public void addEdge(int u, int v) {
        graph.get(u).add(v);
    }

    // Kahn's Algorithm for Topological Sort
    public List<Integer> topologicalSortKahn() {
        // Step 1: Calculate in-degrees
        int[] inDegree = new int[vertices];
        for (int u = 0; u < vertices; u++) {
            for (int v : graph.get(u)) {
                inDegree[v]++;
            }
        }

        // Step 2: Add vertices with in-degree 0 to queue
        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < vertices; i++) {
            if (inDegree[i] == 0) {
                queue.offer(i);
            }
        }

        // Step 3: Process vertices
        List<Integer> result = new ArrayList<>();
        while (!queue.isEmpty()) {
            int current = queue.poll();
            result.add(current);

            // Reduce in-degree of neighbors
```

```java
            for (int neighbor : graph.get(current)) {
                inDegree[neighbor]--;
                if (inDegree[neighbor] == 0) {
                    queue.offer(neighbor);
                }
            }
        }

        // Step 4: Check if topological sort is possible
        if (result.size() != vertices) {
            System.out.println("Graph contains a cycle!");
            return new ArrayList<>();
        }

        return result;
    }

    // Example usage
    public static void main(String[] args) {
        TopologicalSort ts = new TopologicalSort(6);
        ts.addEdge(5, 2);
        ts.addEdge(5, 0);
        ts.addEdge(4, 0);
        ts.addEdge(4, 1);
        ts.addEdge(2, 3);
        ts.addEdge(3, 1);

        List<Integer> result = ts.topologicalSortKahn();
        System.out.println("Topological Sort: " + result);
        // Output: [4, 5, 0, 2, 3, 1] (one possible ordering)
    }
}
```

Time and Space Complexity

- **Time Complexity:** O(V + E) where V is vertices and E is edges
- **Space Complexity:** O(V) for storing in-degrees and queue

2. Detect Cycle in Directed Graph

Why Cycle Detection Matters?

Cycle detection in directed graphs is crucial for:

- Detecting deadlocks in systems
- Validating DAGs before topological sorting

- Finding infinite loops in dependencies

Method 1: DFS-based Cycle Detection

This approach uses three colors to track vertex states:

- **White (0):** Unvisited
- **Gray (1):** Currently being processed (in recursion stack)
- **Black (2):** Completely processed

Key Insight: If we encounter a gray vertex during DFS, we've found a back edge, indicating a cycle.

java

```java
import java.util.*;

public class CycleDetectionDFS {
    private Map<Integer, List<Integer>> graph;
    private int vertices;

    public CycleDetectionDFS(int vertices) {
        this.vertices = vertices;
        this.graph = new HashMap<>();
        for (int i = 0; i < vertices; i++) {
            graph.put(i, new ArrayList<>());
        }
    }

    public void addEdge(int u, int v) {
        graph.get(u).add(v);
    }

    // DFS-based cycle detection
    public boolean hasCycleDFS() {
        // 0: White (unvisited), 1: Gray (processing), 2: Black (processed)
        int[] color = new int[vertices];

        // Check each unvisited vertex
        for (int i = 0; i < vertices; i++) {
            if (color[i] == 0) {
                if (dfsHelper(i, color)) {
                    return true;
                }
            }
        }
        return false;
    }

    private boolean dfsHelper(int vertex, int[] color) {
        // Mark current vertex as being processed (gray)
        color[vertex] = 1;

        // Visit all neighbors
        for (int neighbor : graph.get(vertex)) {
            if (color[neighbor] == 1) {
                // Back edge found - cycle detected
                return true;
            }
            if (color[neighbor] == 0 && dfsHelper(neighbor, color)) {
                return true;
```

```
            }
        }

        // Mark vertex as completely processed (black)
        color[vertex] = 2;
        return false;
    }

    // Example usage
    public static void main(String[] args) {
        CycleDetectionDFS graph = new CycleDetectionDFS(4);
        graph.addEdge(0, 1);
        graph.addEdge(1, 2);
        graph.addEdge(2, 3);
        graph.addEdge(3, 1); // Creates cycle: 1 -> 2 -> 3 -> 1

        if (graph.hasCycleDFS()) {
            System.out.println("Graph contains a cycle");
        } else {
            System.out.println("Graph is acyclic");
        }
    }
}
```

Method 2: BFS-based Cycle Detection (Using Kahn's Algorithm)

We can detect cycles using a modified version of Kahn's algorithm. If we cannot include all vertices in our topological sort, the graph contains a cycle.

java

```java
import java.util.*;

public class CycleDetectionBFS {
    private Map<Integer, List<Integer>> graph;
    private int vertices;

    public CycleDetectionBFS(int vertices) {
        this.vertices = vertices;
        this.graph = new HashMap<>();
        for (int i = 0; i < vertices; i++) {
            graph.put(i, new ArrayList<>());
        }
    }

    public void addEdge(int u, int v) {
        graph.get(u).add(v);
    }

    // BFS-based cycle detection using Kahn's algorithm
    public boolean hasCycleBFS() {
        // Calculate in-degrees
        int[] inDegree = new int[vertices];
        for (int u = 0; u < vertices; u++) {
            for (int v : graph.get(u)) {
                inDegree[v]++;
            }
        }

        // Add vertices with in-degree 0 to queue
        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < vertices; i++) {
            if (inDegree[i] == 0) {
                queue.offer(i);
            }
        }

        // Process vertices
        int processedVertices = 0;
        while (!queue.isEmpty()) {
            int current = queue.poll();
            processedVertices++;

            // Reduce in-degree of neighbors
            for (int neighbor : graph.get(current)) {
                inDegree[neighbor]--;
                if (inDegree[neighbor] == 0) {
```

```
                queue.offer(neighbor);
            }
        }
    }

    // If we couldn't process all vertices, there's a cycle
    return processedVertices != vertices;
}

// Example usage
public static void main(String[] args) {
    CycleDetectionBFS graph = new CycleDetectionBFS(4);
    graph.addEdge(0, 1);
    graph.addEdge(1, 2);
    graph.addEdge(2, 0); // Creates cycle: 0 -> 1 -> 2 -> 0
    graph.addEdge(2, 3);

    if (graph.hasCycleBFS()) {
        System.out.println("Graph contains a cycle");
    } else {
        System.out.println("Graph is acyclic");
    }
}
}
```

Comparison: DFS vs BFS for Cycle Detection

Aspect	DFS Approach	BFS Approach
Time Complexity	O(V + E)	O(V + E)
Space Complexity	O(V)	O(V)
Implementation	Uses recursion/stack	Uses queue
Memory Usage	Can use deep recursion	Iterative, less stack usage
Early Detection	Can stop immediately when cycle found	Must process until queue empty
Cycle Path	Easier to reconstruct cycle path	Harder to get actual cycle

Key Differences Between Directed and Undirected Graphs

Directed Graph Cycles:

- Need to track processing state (gray vertices)
- Back edges indicate cycles
- More complex state management

Undirected Graph Cycles:

- Simply check if we revisit a vertex (except parent)
- Any back edge (except to parent) indicates cycle
- Simpler implementation

Quiz Questions

Question 1: What is the time complexity of Kahn's algorithm for topological sorting? a) O(V) b) O(E) c) O(V + E) d) O(V × E)

Correct Answer: c) O(V + E)

Question 2: In DFS-based cycle detection for directed graphs, what does a "gray" vertex represent? a) An unvisited vertex b) A vertex currently being processed in the recursion stack c) A completely processed vertex d) A vertex with no outgoing edges

Correct Answer: b) A vertex currently being processed in the recursion stack

Question 3: Which of the following statements about topological sort is TRUE? a) It can be performed on any graph b) It's only possible on Directed Acyclic Graphs (DAGs) c) It always produces a unique ordering d) It requires the graph to be connected

Correct Answer: b) It's only possible on Directed Acyclic Graphs (DAGs)

Question 4: In Kahn's algorithm, what happens to vertices with in-degree 0? a) They are ignored b) They are added to a stack c) They are added to a queue for processing d) They are marked as visited

Correct Answer: c) They are added to a queue for processing

Question 5: How can you detect a cycle in a directed graph using BFS? a) If you find a back edge during traversal b) If the number of vertices in topological sort is less than total vertices c) If you encounter a visited vertex d) If the graph is disconnected

Correct Answer: b) If the number of vertices in topological sort is less than total vertices

---

Practice Questions

Question 1: Implement a method that returns all possible topological orderings of a given DAG. The method should return a list of lists, where each inner list represents one valid topological ordering.

Question 2: Given a directed graph representing course prerequisites (where an edge from A to B means A is a prerequisite for B), write a method to determine the minimum number of semesters needed to complete all courses, assuming you can take any number of courses per semester as long as prerequisites are satisfied.

Question 3: Implement a method that not only detects if a cycle exists in a directed graph but also returns one of the cycles if it exists. The cycle should be returned as a list of vertices forming the cycle.

Question 4: Write a program to find the longest path in a Directed Acyclic Graph (DAG). You can use the concept of topological sorting to solve this efficiently.

Question 5: Given a directed graph, implement a method to find all strongly connected components and determine if removing a specific edge would eliminate all cycles in the graph.